NutriTrack

Food Nutrition Tracking & Analysis API

Chenxi Li

XJCO3011 Web Services and Web Data — Coursework 1

April 2026

Live: https://lichenxi.pythonanywhere.com

People lack programmable tools to track nutrition — NutriTrack bridges this gap

Problem

- Tracking daily nutritional intake is complex and tedious
- Existing tools are either too complex or not programmable
- AI assistants cannot directly interact with nutrition databases

Solution: NutriTrack

- **REST API built on USDA SR Legacy dataset**
  - 7,793 foods across 25 categories
- Food search with relevance ranking
- Meal logging with full CRUD
- Nutritional analytics (daily/weekly/balance)
- MCP integration → AI can call the API

FastAPI + SQLite + MCP enables a lightweight yet fully-featured nutrition API

Component	Choice	Rationale
Framework	FastAPI	Auto Swagger docs, Pydantic validation, lightweight
Database	SQLite	Zero-config, portable, sufficient for ~7,800 records
ORM	SQLAlchemy 2.0	Alembic migrations, complex query support
Auth	JWT (python-jose)	Stateless, RESTful-aligned
MCP	FastMCP SDK	Official Python SDK, stdio transport

HTTP Clients / MCP Server → FastAPI (17 endpoints) → SQLite (7,793 foods + users + meals)

Auth (JWT) · Foods (search/categories) · Meals (CRUD) · Analytics (daily/weekly/balance) · User Profile

Smart search, two-tier validation, and personalised analysis are the key design decisions

4 Data Models

User → Meal → MealItem → Food
Relational chain with foreign keys

Smart Search

3-level relevance ranking:

starts-with > word-boundary > substring

Optional category filter (25 categories)

Two-Tier Input Validation

Hard reject: impossible values (negative weight, future dates)

Soft warning: "Weight 130 kg is unusually high. Please verify."

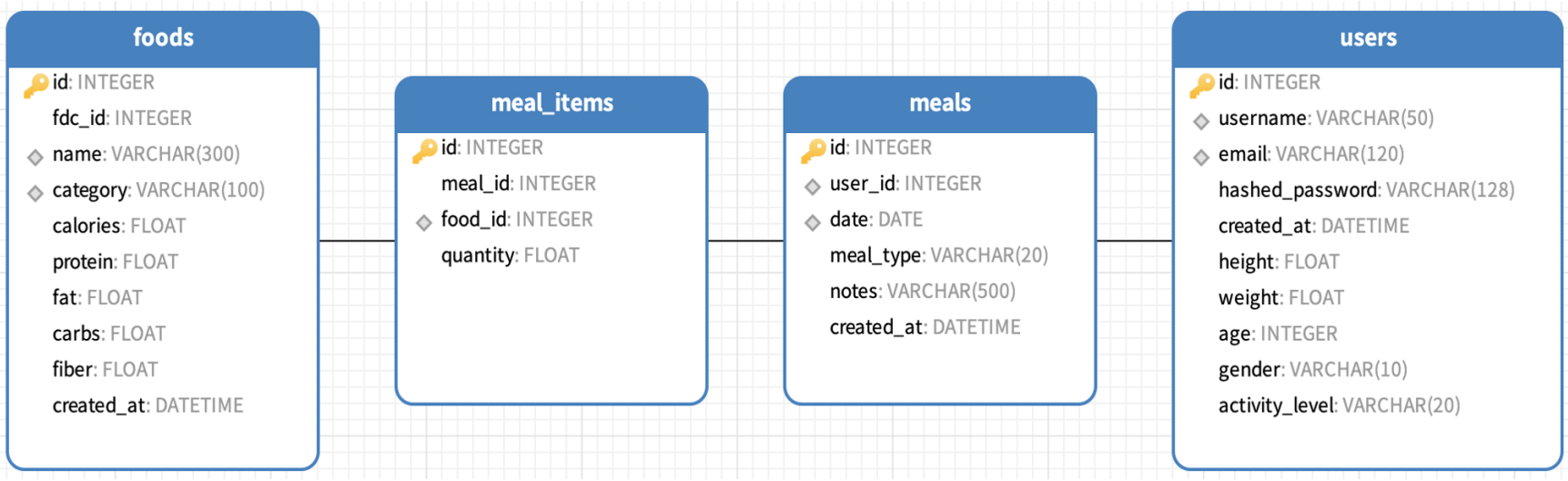
Personalised Nutrition

Mifflin-St Jeor equation for BMR

Adjusts for age, gender, height, weight, activity level

Balance analysis: low / ok / high per nutrient

Database ER Diagram



Relationships: User 1 - N Meal (one user has many meals) · Meal 1 - N MealItem (one meal contains many items) · Food 1 - N MealItem (one food appears in many items) · **Food N - N Meal** (many-to-many via MealItem junction table)

MCP integration enables AI assistants to call all 16 API tools via natural language

What is MCP?

- Model Context Protocol — open standard by Anthropic
- Allows AI assistants to call external tools via structured interface
- stdio transport, works with Claude Desktop

Our MCP Server

16 tools covering all API endpoints

- Full CRUD: search, log, update, delete meals
- Analytics: daily summary, weekly trends, balance
- User management: register, login, profile

Example Flow

User: *"Log a lunch with chicken breast and rice"*
→ AI calls search_food("chicken breast") → log_meal({items: [...]}) → returns confirmation

Live demo shows AI-powered meal logging through MCP + 17 Swagger endpoints

MCP Demo (video clip)

1. AI searches for food → search_food tool
2. AI logs a meal → log_meal tool
3. AI shows summary → get_daily_summary tool

Claude Desktop discovers all 16 tools automatically

MCP Video:
OneDrive (login required): Nutritrack MCP Test.mp4
Google Drive: Nutritrack MCP Test.mp4
Baidu Pan: Nutritrack MCP Test.mp4 (Password: q3md)

Swagger UI — 17 Endpoints

Auth: register, login

Foods: search, categories, by-category, by-id

Meals: create, list, detail, update, delete

Analytics: daily summary, weekly trend, balance

User: profile, update profile

Online API Doc:
Swagger UI Live:https://lichenxi.pythonanywhere.com/docs
ReDoc:https://lichenxi.pythonanywhere.com/redoc

Version Control & Testing

Git History (16 commits)

d0c475d Initial commit

27f54a1 feat: project skeleton, models, alembic migration, USDA import

4a7d494 feat: JWT auth, Food read API, Meal CRUD

6924269 feat: nutrition analytics (daily/weekly/balance)

70637d4 feat: user profile + personalized nutrition + MCP tools

1b744b7 feat: input validation — hard reject + soft warnings

6789771 feat: search improvements + MCP demo assets

f0b77d1 refactor: async → sync for PythonAnywhere WSGI

82f5857 test: 28 integration tests with configurable base URL

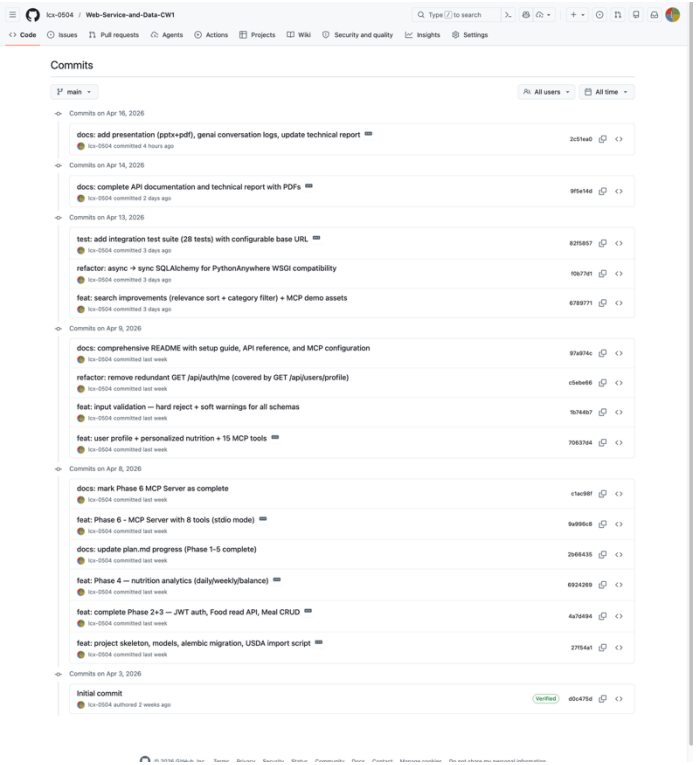
9f5e14d + 2c51ea0 docs: API docs, tech report, presentation, genai logs

Testing (28 integration tests)

Auth, Foods, Meals CRUD, Analytics, User Profile

Configurable: local (127.0.0.1:8000) or remote (PythonAnywhere)

Edge cases: duplicate registration, unauthorized access, future dates



Links & Deliverables

Deliverable	Link / Location
Code Repository	https://github.com/lcx-0504/Web-Service-and-Data-CW1
Commit History (16 commits)	https://github.com/lcx-0504/Web-Service-and-Data-CW1/commits/main
Live API (Deployed on PythonAnywhere)	https://lichenxi.pythonanywhere.com/
Swagger UI	https://lichenxi.pythonanywhere.com/docs
ReDoc	https://lichenxi.pythonanywhere.com/redoc
API Documentation (PDF – 17 Endpoints)	https://github.com/lcx-0504/Web-Service-and-Data-CW1/blob/main/docs/api-documentation.pdf
Technical Report (PDF)	https://github.com/lcx-0504/Web-Service-and-Data-CW1/blob/main/docs/technical-report.pdf
Presentation (PPTX)	https://github.com/lcx-0504/Web-Service-and-Data-CW1/blob/main/docs/presentation.pdf
GenAI Logs (Claude Code & Github Copilot with Claude Opus 4.6)	https://github.com/lcx-0504/Web-Service-and-Data-CW1/tree/main/docs/genai-logs
MCP Server (16 tools)	https://github.com/lcx-0504/Web-Service-and-Data-CW1/blob/main/mcp_server/server.py
MCP Demo Video	Multiple Links Provided in README: https://github.com/lcx-0504/Web-Service-and-Data-CW1/tree/main?tab=readme-ov-file#mcp-demo-video
Integration Tests	Test Script: https://github.com/lcx-0504/Web-Service-and-Data-CW1/blob/main/backend/tests/test_api.py Test Log: https://github.com/lcx-0504/Web-Service-and-Data-CW1/blob/main/docs/test-results.txt

Technical Report Summary & Conclusions

1. Fully tested: 28 integration tests across 5 modules — all passed against PythonAnywhere.

2. Deployed: PythonAnywhere free tier with custom ASGI→WSGI bridge (3 iterations to solve).

3. MCP-enabled: 16 tools allow AI assistants to interact with the entire API via natural language.

4. GenAI-assisted: GitHub Copilot + Claude Code (Opus 4.6) for planning, coding, debugging, and docs.

Chenxi Li · GitHub: github.com/lcx-0504/Web-Service-and-Data-CW1

Thank you — Questions?